

Time Series Analysis with Python

Comprehensive Practical Exercise: Decomposition, Moving Average, Stationarity, ADF, ACF, PACF, ARIMA, and SARIMA

Qiyas Training | Time Series Analysis - Lab 1 | Instructor: Yadu

1. Learning Objectives

By the end of this exercise, you should be able to:

1. Load and inspect time-series data.
2. Visualize trends and seasonality.
3. Calculate and interpret moving averages.
4. Understand additive and multiplicative decomposition.
5. Identify whether a series is stationary.
6. Perform and interpret the Augmented Dickey-Fuller (ADF) test.
7. Difference a non-stationary series.
8. Interpret ACF and PACF plots.
9. Build an ARIMA model.
10. Build a SARIMA model.
11. Compare ARIMA and SARIMA.
12. Evaluate forecasting performance on unseen data.
13. Select a suitable model based on evidence rather than simply choosing an arbitrary model.

2. Dataset

For this exercise, use a synthetic monthly retail sales dataset. The dataset contains:

- `date` : monthly date
- `sales` : monthly sales
- `customers` : number of customers
- `promotion` : whether a promotion was active

The dataset intentionally contains **Trend**, **Seasonality**, **Random noise**, and **Increasing seasonal variation**. This makes it useful for understanding additive versus multiplicative decomposition.

Exercise 0: Create the Dataset

Task: Run the following code to create the dataset.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

np.random.seed(42)

dates = pd.date_range(
    start="2018-01-01",
    end="2025-12-01",
    freq="MS"
)

n = len(dates)

trend = np.linspace(100, 250, n)

seasonal_pattern = np.array([
    0.80, 0.85, 0.90, 0.95,
    1.00, 1.05, 1.10, 1.05,
    1.00, 1.10, 1.25, 1.50
])

seasonality = np.tile(seasonal_pattern, n // 12)

noise = np.random.normal(0, 8, n)

sales = trend * seasonality + noise

customers = (
    sales * 4
    + np.random.normal(0, 30, n)
)

promotion = np.random.choice(
    [0, 1],
    size=n,
    p=[0.7, 0.3]
)

sales = sales + promotion * 15

df = pd.DataFrame({
    "date": dates,
    "sales": sales,
    "customers": customers,
    "promotion": promotion
})

df.head()
```

Convert the date column and make it the index:

```
df["date"] = pd.to_datetime(df["date"])
df.set_index("date", inplace=True)
df.head()
```

Questions

1. How many observations are in the dataset?
2. What is the frequency of the data?
3. What variables are available?
4. What does the `promotion` variable represent?

Part 1: Visualizing the Time Series

Exercise 2: Plot Monthly Sales

Create a line plot of monthly sales:

```
plt.figure(figsize=(14, 5))
plt.plot(df.index, df["sales"])
plt.title("Monthly Retail Sales")
plt.xlabel("Date")
plt.ylabel("Sales")
plt.show()
```

Questions

Look carefully at the graph:

1. Is there a trend?
2. Is there seasonality?
3. Are there unusually high or low observations?
4. Does the size of seasonal fluctuations appear constant?
5. Can you identify months that repeatedly have higher sales?

Hint: Do not calculate anything yet. First learn to visually inspect a time series.

Part 2: Moving Average

A moving average smooths short-term fluctuations so that the underlying pattern becomes easier to see.

Exercise 3: Calculate a 3-Month Moving Average

Create a 3-month moving average and plot the original sales alongside it:

```
df["MA_3"] = df["sales"].rolling(window=3).mean()

plt.figure(figsize=(14, 5))
plt.plot(df.index, df["sales"], label="Actual Sales")
plt.plot(df.index, df["MA_3"], label="3-Month Moving Average")
plt.legend()
plt.title("Sales and 3-Month Moving Average")
plt.show()
```

Questions

1. What does the moving average do to the series?
2. Is the moving average smoother than the original series?
3. Why are the first observations missing?
4. Does the moving average remove some random fluctuations?
5. Does it completely remove seasonality?

Hint: A 3-month moving average needs three observations to calculate its first value.

Exercise 4: Compare Different Moving Averages

Calculate multi-window moving averages and plot them together:

```
df["MA_3"] = df["sales"].rolling(3).mean()
df["MA_6"] = df["sales"].rolling(6).mean()
df["MA_12"] = df["sales"].rolling(12).mean()

plt.figure(figsize=(14, 6))
plt.plot(df.index, df["sales"], label="Actual Sales")
plt.plot(df.index, df["MA_3"], label="MA 3")
plt.plot(df.index, df["MA_6"], label="MA 6")
plt.plot(df.index, df["MA_12"], label="MA 12")
plt.legend()
plt.title("Comparison of Moving Averages")
plt.show()
```

Questions

Compare the 3-month, 6-month, and 12-month moving averages:

1. Which window is most responsive to recent changes?
2. Which window produces the smoothest curve?
3. Which window removes the most short-term variation?
4. Which one would be better for identifying the long-term trend?
5. What is the trade-off between a small and large window?

Key Idea:

Small window → More responsive, More noise
Large window → More smoothing, Less responsive

Part 3: Additive and Multiplicative Decomposition

Time-series decomposition separates a series into components: **Observed**, **Trend**, **Seasonal**, and **Residual**.

Exercise 5: Additive Decomposition

The additive model assumes: $\text{Observed} = \text{Trend} + \text{Seasonality} + \text{Residual}$

```
from statsmodels.tsa.seasonal import seasonal_decompose

additive = seasonal_decompose(
    df["sales"],
    model="additive",
    period=12
)

additive.plot()
plt.show()
```

Questions

Examine the four components (Observed, Trend, Seasonal, Residual):

1. What is happening to the trend?
2. What is the seasonal pattern?
3. Is the seasonal effect approximately constant?
4. What does the residual represent?
5. Which months appear to have positive seasonal effects?
6. Which months appear to have negative seasonal effects?

Hint: The dataset is monthly, so one year corresponds to 12 months. Therefore: `period=12` .

Exercise 6: Multiplicative Decomposition

The multiplicative model assumes: $\text{Observed} = \text{Trend} \times \text{Seasonality} \times \text{Residual}$

```
multiplicative = seasonal_decompose(
    df["sales"],
    model="multiplicative",
    period=12
)

multiplicative.plot()
plt.show()
```

Questions

1. How does the multiplicative decomposition differ from the additive decomposition?
2. Does seasonal variation increase as sales increase?
3. Is the seasonal pattern proportional to the level of the series?
4. Does the multiplicative decomposition appear reasonable for this dataset?

Exercise 7: Additive vs Multiplicative Comparison

Complete the following table in your notes:

Characteristic	Additive	Multiplicative
Basic relationship	?	?
Seasonal effect	?	?
Seasonal variation constant?	?	?
Seasonal variation grows with level?	?	?
Example	?	?

Analytical Prompts:

- **Question 1:** When should we use additive decomposition?
- **Question 2:** When should we use multiplicative decomposition?

Hint — Compare:

Additive: Jan = $100 + 20$, Feb = $110 + 20$, Mar = $120 + 20$ (Seasonal effect is constant $\sim +20$)

Multiplicative: Jan = 100×1.20 , Feb = 110×1.20 , Mar = 120×1.20 (Seasonal effect grows with level)

Part 4: Stationarity

A stationary time series has statistical properties that are relatively stable over time. In practical terms, examine: Mean, Variance, Trend, Seasonality, and Dependence structure.

Exercise 8: Visual Stationarity Assessment

Look again at the original sales series.

Questions

1. Does the mean appear constant over time?
2. Does the variance appear constant?
3. Does the series have a trend?
4. Does the series have seasonality?
5. Would you classify the original sales series as stationary or non-stationary?
6. Explain your answer using evidence from the graph.

Important: Do not use the ADF test yet. First make your decision based on visual inspection.

Exercise 9: Compare Different Periods

Split the series into three periods and compute summary metrics:

```
first = df["sales"].iloc[:24]
middle = df["sales"].iloc[24:48]
last = df["sales"].iloc[-24:]

print("First period mean:", first.mean())
print("Middle period mean:", middle.mean())
print("Last period mean:", last.mean())

print("First period std:", first.std())
print("Middle period std:", middle.std())
print("Last period std:", last.std())
```

Questions

1. Is the mean approximately constant?
2. Is the standard deviation approximately constant?
3. What does this tell you about stationarity?
4. Why is visual inspection alone not always sufficient?

Part 5: Augmented Dickey-Fuller (ADF) Test

The ADF test determines whether a time series contains a unit root:

H_0 : Unit root exists $\rightarrow$ Series is non-stationary

H_1 : No unit root $\rightarrow$ Series is stationary

Decision Rule:

p-value $< 0.05 \rightarrow$ Reject $H_0 \rightarrow$ Stationary

p-value $> 0.05 \rightarrow$ Fail to reject $H_0 \rightarrow$ Non-stationary

Exercise 10: Perform the ADF Test

```
from statsmodels.tsa.stattools import adfuller

result = adfuller(df["sales"])

print("ADF Statistic:", result[0])
print("p-value:", result[1])

for key, value in result[4].items():
    print(f"Critical Value ({key}): {value}")
```

Questions

1. What is the ADF statistic?
2. What is the p-value?
3. What is the null hypothesis?
4. Do you reject or fail to reject the null hypothesis?
5. Is the original sales series stationary?

Exercise 11: Interpret ADF Results

Suppose you obtain: ADF Statistic: -1.50, p-value: 0.53.

Questions

1. Is $0.53 < 0.05$?
2. Should H_0 be rejected?
3. Does the result provide evidence of stationarity?
4. What should you do next?

Important Rule to Memorize:

- ADF p-value < 0.05 —► Reject H_0 —► **Stationary**
- ADF p-value > 0.05 —► Fail to reject H_0 —► **Non-stationary**

Part 6: Differencing

Differencing is commonly used to remove trend and help make a series stationary. First difference: $Y(t) - Y(t-1)$.

Exercise 12: First-Order Differencing

```
df["sales_diff"] = df["sales"].diff()

plt.figure(figsize=(14, 5))
plt.plot(df.index, df["sales_diff"])
plt.title("First Differenced Sales")
plt.xlabel("Date")
plt.ylabel("Difference")
plt.show()
```

Questions

1. Did differencing reduce the trend?
2. Does the series look more stable?
3. What happened to the first observation?
4. Why is the first observation missing?
5. Does seasonality still appear?

Exercise 13: ADF Test After Differencing

```
result_diff = adfuller(df["sales_diff"].dropna())

print("ADF Statistic:", result_diff[0])
print("p-value:", result_diff[1])

for key, value in result_diff[4].items():
    print(f"Critical Value ({key}): {value}")
```

Questions

1. Did the p-value change?
2. Is the differenced series stationary?
3. How many differences were required?
4. What value of `d` would you initially consider for ARIMA?

Part 7: Autocorrelation Function (ACF)

ACF measures how strongly the current value is related to values from previous time steps (Lag 1 = 1 step prior, Lag 12 = 12 steps prior).

Exercise 14: Plot ACF

```
from statsmodels.graphics.tsaplots import plot_acf

plot_acf(
    df["sales_diff"].dropna(),
    lags=30
)
plt.show()
```

Questions

1. What does lag 1 represent?
2. What does lag 2 represent?
3. What does lag 12 represent?
4. Which lags appear significant?
5. Is there evidence of seasonal autocorrelation?
6. Does the ACF cut off quickly or tail off gradually?

Hint: Look at the confidence bands (blue shaded area) in the plot.

Part 8: Partial Autocorrelation Function (PACF)

PACF measures the relationship between the current value and a particular lag after accounting for the effects of intermediate lags.

Exercise 15: Plot PACF

```
from statsmodels.graphics.tsaplots import plot_pacf

plot_pacf(
    df["sales_diff"].dropna(),
    lags=30,
    method="ywm"
)
plt.show()
```

Questions

1. Which lags appear significant?
2. Does PACF cut off quickly?
3. Does PACF tail off?
4. What AR order might you initially consider?
5. Is there evidence of seasonal dependence?

Part 9: ARIMA Models

ARIMA stands for **AutoRegressive Integrated Moving Average**: $ARIMA(p, d, q)$ where p = AR order, d = differencing order, and q = MA order.

Exercise 16: Train-Test Split

Do not train your model using the entire dataset. Use the last 12 months as the test set:

```
train = df["sales"].iloc[:-12]
test = df["sales"].iloc[-12:]

print("Training observations:", len(train))
print("Testing observations:", len(test))
```

Questions

1. Why should we keep a test set?
2. Why should time-series data not normally be randomly shuffled before splitting?
3. Why are the most recent observations useful as a test set?

Exercise 17: Build Your First ARIMA Model

```
from statsmodels.tsa.arima.model import ARIMA

model = ARIMA(
    train,
    order=(1, 1, 1)
)

model_fit = model.fit()
print(model_fit.summary())
```

Structure Breakdown:

```
ARIMA(1, 1, 1)
|  |  |
|  |  └─ MA = 1 (Moving Average order q)
|  └─ Differencing = 1 (Integration order d)
└─ AR = 1 (Autoregressive order p)
```

Questions

1. What does `p=1` mean?
2. What does `d=1` mean?
3. What does `q=1` mean?
4. Why did we use `d=1` ?
5. How did ACF and PACF help us choose candidate values for `p` and `q` ?

Exercise 18: Forecast the Test Period

```
forecast = model_fit.forecast(steps=12)
print(forecast)

plt.figure(figsize=(14, 5))
plt.plot(train.index, train, label="Training")
plt.plot(test.index, test, label="Actual")
plt.plot(test.index, forecast, label="Forecast")
plt.legend()
plt.title("ARIMA Forecast")
plt.show()
```

Questions

1. Does the forecast follow the general direction of the actual series?
2. Does it capture the seasonal behavior?
3. Where does the model make large errors?
4. What does ARIMA appear to struggle with?

Important Observation: Ordinary ARIMA does not explicitly model seasonal behavior. This motivates SARIMA.

Part 10: ARIMA Evaluation

Exercise 19: Calculate MAE and RMSE

```
from sklearn.metrics import mean_absolute_error, mean_squared_error
import numpy as np

mae = mean_absolute_error(test, forecast)
print("MAE:", mae)

rmse = np.sqrt(mean_squared_error(test, forecast))
print("RMSE:", rmse)
```

Questions

1. What does MAE measure?
2. What does RMSE measure?
3. Which metric is more sensitive to large errors?
4. Is a lower or higher MAE better?
5. Is a lower or higher RMSE better?

Part 11: SARIMA Models

SARIMA extends ARIMA by explicitly modeling seasonal behavior: $\text{SARIMA}(p, d, q)(P, D, Q, s)$ where p, d, q are non-seasonal orders, P, D, Q are seasonal orders, and s is the seasonal period (for monthly data, $s=12$).

Exercise 20: Build a SARIMA Model

```
from statsmodels.tsa.statespace.sarimax import SARIMAX

model = SARIMAX(
    train,
    order=(1, 1, 1),
    seasonal_order=(1, 1, 1, 12)
)

model_fit = model.fit(dispatch=False)
print(model_fit.summary())
```

SARIMA(1,1,1)(1,1,1,12) Breakdown:

- Non-seasonal: AR = 1, Difference = 1, MA = 1
- Seasonal: Seasonal AR = 1, Seasonal Difference = 1, Seasonal MA = 1
- Seasonal period: 12 months

Questions

1. What does the first $(1,1,1)$ represent?
2. What does $(1,1,1,12)$ represent?
3. Why is the seasonal period 12?
4. Why might SARIMA be more appropriate than ARIMA for this dataset?

Exercise 21: Forecast with SARIMA

```
forecast_sarima = model_fit.forecast(steps=12)

plt.figure(figsize=(14, 5))
plt.plot(train.index, train, label="Training")
plt.plot(test.index, test, label="Actual")
plt.plot(test.index, forecast_sarima, label="SARIMA Forecast")
plt.legend()
plt.title("SARIMA Forecast")
plt.show()
```

Questions

1. Does SARIMA capture seasonal behavior better than ARIMA?
2. Does the forecast follow the actual data?
3. Where does the model make large errors?

Exercise 22: Evaluate SARIMA

```
mae_sarima = mean_absolute_error(test, forecast_sarima)
rmse_sarima = np.sqrt(mean_squared_error(test, forecast_sarima))

print("SARIMA MAE:", mae_sarima)
print("SARIMA RMSE:", rmse_sarima)
```

Exercise 23: Compare ARIMA and SARIMA

```
comparison = pd.DataFrame({
    "Model": ["ARIMA", "SARIMA"],
    "MAE": [mae, mae_sarima],
    "RMSE": [rmse, rmse_sarima]
})
print(comparison)
```

Questions

1. Which model has lower MAE?
2. Which model has lower RMSE?
3. Which model better captures seasonality?
4. Why might SARIMA outperform ARIMA?
5. Does a more complicated model always guarantee better forecasting?
6. What would happen if the dataset had no seasonality?

Part 12: Model Experimentation

At this point, stop simply following the examples. Experiment with multiple parameter configurations.

Exercise 24: Try Different ARIMA Models

Test candidate specifications: $ARIMA(0, 1, 1)$, $ARIMA(1, 1, 0)$, $ARIMA(1, 1, 1)$, $ARIMA(2, 1, 1)$, $ARIMA(1, 1, 2)$, $ARIMA(2, 1, 2)$.

For each model: (1) Fit the model, (2) Forecast 12 months, (3) Calculate MAE & RMSE, (4) Record results.

Model	MAE	RMSE
ARIMA(0,1,1)		
ARIMA(1,1,0)		
ARIMA(1,1,1)		
ARIMA(2,1,1)		
ARIMA(1,1,2)		
ARIMA(2,1,2)		

Forecasting Accuracy Metric Goal: Lower MAE —► Better | Lower RMSE —► Better

Exercise 25: Try Different SARIMA Models

Experiment with:

- $SARIMA(1, 1, 1)(1, 1, 1, 12)$
- $SARIMA(0, 1, 1)(1, 1, 1, 12)$
- $SARIMA(1, 1, 0)(1, 1, 1, 12)$
- $SARIMA(1, 1, 1)(0, 1, 1, 12)$

Model	AIC	MAE	RMSE
SARIMA 1			
SARIMA 2			
SARIMA 3			
SARIMA 4			

Questions

1. Which model has the lowest AIC?
2. Which has the lowest MAE?
3. Which has the lowest RMSE?
4. Do the same models always win on all metrics?
5. Which model would you choose and why?

Part 13: Final Forecasting Challenge

Now build a final forecasting model trained on the complete dataset.

Exercise 26: Train the Best Model on All Historical Data

Replace the ... placeholders with the best parameters identified:

```
final_model = SARIMAX(
    df["sales"],
    order=(..., ..., ...),
    seasonal_order=(..., ..., ..., 12)
)

final_fit = final_model.fit(dispatch=False)
```

Exercise 27: Forecast the Next 12 Months

```
future_forecast = final_fit.forecast(steps=12)

future_dates = pd.date_range(
    start=df.index[-1] + pd.DateOffset(months=1),
    periods=12,
    freq="MS"
)

future = pd.DataFrame({
    "date": future_dates,
    "forecast_sales": future_forecast
})
print(future)
```

Exercise 28: Plot the Final Forecast

```
plt.figure(figsize=(14, 5))
plt.plot(df.index, df["sales"], label="Historical Sales")
plt.plot(future["date"], future["forecast_sales"], label="Future Forecast")
plt.legend()
plt.title("12-Month Sales Forecast")
plt.show()
```

Questions

1. What is the predicted sales for each future month?
2. Which future month has the highest predicted sales?
3. Which future month has the lowest predicted sales?
4. Can you identify a seasonal pattern in the forecast?
5. Does the forecast make business sense?

Part 14: Final Analysis & Synthesis

Exercise 29: Write Your Conclusions

Answer all the following questions using evidence from your lab execution:

A. Decomposition

1. What trend did you observe?
2. What seasonal pattern did you observe?
3. Was additive or multiplicative decomposition more appropriate?
4. Why?

B. Moving Average

1. What is a moving average?
2. What happens when the window size increases?
3. What is the trade-off between smoothing and responsiveness?
4. Which moving average was most useful for understanding the trend?

C. Stationarity

1. Was the original sales series stationary?
2. How did you determine this?
3. What did differencing do?
4. What value of `d` did you use?

D. ADF

1. What is the null hypothesis of the ADF test?
2. What does a p-value below 0.05 mean?
3. What does a p-value above 0.05 mean?
4. What was your ADF conclusion before differencing?
5. What was your ADF conclusion after differencing?

E. ACF and PACF

1. What does ACF measure?
2. What does PACF measure?
3. What is the difference between ACF and PACF?
4. How did ACF help you select `q`?
5. How did PACF help you select `p`?
6. Did you observe seasonal autocorrelation?

F. ARIMA

1. What do `p`, `d`, and `q` represent?
2. Which ARIMA model performed best?
3. What were its MAE and RMSE?
4. Did ARIMA capture seasonality well?

G. SARIMA

1. Why do we use SARIMA?
2. What do `P`, `D`, `Q`, and `s` represent?
3. Why was `s=12` used?

4. Did SARIMA outperform ARIMA?

5. Why or why not?

H. Final Forecast

1. Which model did you select for the final forecast?

2. Why did you select it?

3. What are the predicted sales for the next 12 months?

4. What business decisions could be made using these forecasts?

Part 15: Quick Reference Sheet

Moving Average

Smooths short-term fluctuations.

- **Small window:** More responsive, More noise
- **Large window:** More smoothing, Less responsive

Additive vs Multiplicative Decomposition

- **Additive:** $\text{Observed} = \text{Trend} + \text{Seasonality} + \text{Residual}$ (Use when seasonal fluctuations are approximately constant)
- **Multiplicative:** $\text{Observed} = \text{Trend} \times \text{Seasonality} \times \text{Residual}$ (Use when seasonal fluctuations grow with the level of the series)

Stationarity & ADF Test

A stationary series has relatively stable mean, variance, and autocorrelation structure.

- H_0 : Unit root exists $\rightarrow$ Non-stationary
- H_1 : No unit root $\rightarrow$ Stationary
- $p < 0.05 \rightarrow$ Reject $H_0 \rightarrow$ Stationary
- $p > 0.05 \rightarrow$ Fail to reject $H_0 \rightarrow$ Non-stationary

ACF & PACF Diagnostics

- **ACF (Autocorrelation Function):** Measures correlation between current value and previous lags. Identifies candidate **MA(q)** terms.
- **PACF (Partial Autocorrelation Function):** Measures direct relationship with a lag after accounting for intermediate lags. Identifies candidate **AR(p)** terms.

ARIMA & SARIMA Specifications

- **ARIMA(p, d, q):** p = AR order, d = Differencing order, q = MA order
- **SARIMA(p, d, q)(P, D, Q, s):**
 - p, d, q = Non-seasonal AR, differencing, and MA
 - P, D, Q = Seasonal AR, differencing, and MA
 - s = Seasonal period (e.g. $s = 12$ for monthly data)